Sprint 18 Structured Notes: Debugging, Errors, and Developer Tools

0. What Sprint 18 Is About

Sprint 18 is about learning how to find and fix problems in JavaScript code.

Before this sprint, you learned how to build beginner browser apps using:

- values
- variables
- strings
- numbers
- booleans
- conditions
- functions
- arrays
- loops
- objects
- callbacks
- Sets and Maps
- DOM selection
- DOM updates
- events
- accessibility state
- forms and validation
- dates, time, audio, video, and media APIs
- CRUD
- browser storage
- `localStorage`
- `sessionStorage`
- `JSON.stringify()`
- `JSON.parse()`

Now your apps are becoming more realistic. Realistic apps often break while you are building them.

Sprint 18 teaches you how to understand those problems instead of guessing.

Main goal:

> Learn how to diagnose JavaScript problems in a calm, systematic way.

By the end of Sprint 18, you should understand:

- debugging
- JavaScript error messages
- error names
- error messages
- file names and line numbers
- `SyntaxError`
- `ReferenceError`
- `TypeError`
- `RangeError`
- `throw`
- `new Error()`
- `new TypeError()`
- `new RangeError()`
- `try`
- `catch`
- `finally`
- the `error` object
- `error.name`
- `error.message`
- `error.stack`
- `debugger`
- browser DevTools
- breakpoints
- watch expressions
- `console.log()`
- `console.error()`
- `console.table()`
- `console.assert()`
- safe error handling
- beginner testing habits
- how to debug a broken function
- how to debug wrong data types
- how to debug stored browser data

---

1. Current Progress Before Sprint 18

Sprint 18 comes after you have already learned the main JavaScript and browser basics.

1.1 Core JavaScript Progress

You should now be able to work with:

Area	What You Should Know
Values and variables	Store data with `let` and `const`.
Data types	Recognize strings, numbers, booleans, objects, arrays, `null`, and `undefined`.
Strings	Create, search, slice, trim, replace, and format text.
Numbers and booleans	Calculate, compare, and make decisions.
Functions	Write reusable logic with parameters and return values.
Arrays	Store ordered lists of data.
Loops	Repeat work safely.
Objects	Store structured key-value data.
Callbacks	Pass functions into other functions.
Sets and Maps	Use special collections for unique values and key-value pairs.

1.2 Browser JavaScript Progress

You should now be able to:

Area	What You Should Know
DOM selection	Find page elements with JavaScript.
DOM updates	Change text, attributes, classes, styles, and children.
Events	Respond to clicks, typing, form submission, and timers.
Accessibility	Keep visual state and assistive-technology state in sync.
Forms	Validate user input and control submission.
Dates and media	Use browser APIs for time, audio, video, and media features.
Storage	Save simple data with `localStorage` and `sessionStorage`.

1.3 Why Sprint 18 Comes Here

Before Sprint 18, you learned how to build small apps.

Example:

```
let tasks = [];
```

```
tasks.push({
  text: "Study JavaScript",
  done: false
});

localStorage.setItem("tasks", JSON.stringify(tasks));
```

This code uses several earlier ideas:

- arrays
- objects
- strings
- browser storage
- JSON

But realistic code can fail.

For example:

```
const savedTasks = JSON.parse(localStorage.getItem("tasks"));
```

This can fail if the stored data is broken.

That is why debugging matters.

Simple summary:

> Sprint 17 taught you how to build apps that remember data. Sprint 18 teaches you how to investigate problems when those apps break.

---

2. What Is Debugging?

Debugging means finding and fixing problems in code.

A problem in code is often called a bug.

Example:

```
const price = "abc";
const quantity = 3;

const total = price * quantity;
```

```
console.log(total);
```

Output:

```
NaN
```

The code runs, but the result is wrong.

Why?

Because `price` is not a valid number.

Debugging means asking clear questions:

1. What did I expect to happen?
2. What actually happened?
3. Which value is wrong?
4. Where did the value become wrong?
5. What should I change?

Beginner rule:

> Debugging is investigation, not guessing.

---

3. The Basic Debugging Mindset

When code breaks, do not randomly change lines.

Use a process.

3.1 A Good Debugging Process

Follow this order:

1. Read the error message.
2. Find the line number.
3. Check the value at that line.
4. Check the value's type.
5. Check whether the value is `null` or `undefined`.
6. Print useful values with `console.log()`.
7. Use `console.table()` for arrays of objects.
8. Pause the code with `debugger` or breakpoints.
9. Fix one problem at a time.

10. Test the fix.

3.2 What Not to Do

Avoid this approach:

```
Change random code.
Run again.
Change more random code.
Run again.
Hope it works.
```

That is not reliable.

Better approach:

```
Read.
Inspect.
Test.
Fix.
Verify.
```

---

4. How to Read a JavaScript Error Message

When JavaScript shows an error, it usually gives you useful clues.

Example:

```
TypeError: users.map is not a function
    at app.js:12
```

Read it in three parts.

Part	Example	Meaning
Error name	`TypeError`	The kind of problem.
Error message	`users.map is not a function`	What JavaScript is complaining about.
File and line number	`app.js:12`	Where the problem happened.

4.1 Error Name

The error name tells you the category of the problem.

Examples:

```
SyntaxError
ReferenceError
TypeError
RangeError
```

4.2 Error Message

The message gives more detail.

Example:

```
users.map is not a function
```

This means JavaScript tried to call `.map()`, but the value did not have a valid `.map()` method.

Most likely, `users` is not an array.

4.3 File and Line Number

The line number tells you where to look first.

Example:

```
app.js:12
```

This means:

> Look in `app.js` near line 12.

Beginner rule:

> Red console text is not just failure. It is information.

---

5. Common JavaScript Error Types

Sprint 18 focuses on four common JavaScript errors:

1. `SyntaxError`
2. `ReferenceError`
3. `TypeError`
4. `RangeError`

---

6. `SyntaxError`

A `SyntaxError` means JavaScript cannot understand the structure of your code.

Simple meaning:

> The code grammar is invalid.

6.1 Example: Missing Parenthesis

Incorrect:

```
console.log("Hello";
```

Problem:

```
// The closing parenthesis is missing.
```

Correct:

```
console.log("Hello");
```

6.2 Example: Missing Curly Brace

Incorrect:

```
if (true) {
  console.log("Hi");
```

Problem:

```
// The closing } is missing.
```

Correct:

```
if (true) {
  console.log("Hi");
}
```

6.3 Example: Missing Quote

Incorrect:

```
const name = "Ada;
```

Problem:

```
// The closing quote is missing.
```

Correct:

```
const name = "Ada";
```

6.4 Example: Extra Comma

Incorrect:

```
const user = {
  name: "Ada",
  age: 25,,
};
```

Correct:

```
const user = {
  name: "Ada",
  age: 25,
};
```

6.5 Why `SyntaxError` Is Important

A `SyntaxError` often stops the script before it runs.

That means your program may not execute at all until the syntax is fixed.

Beginner rule:

> If JavaScript cannot read the code, it cannot run the code.

---

7. `ReferenceError`

A `ReferenceError` happens when JavaScript cannot find a variable or name.

Simple meaning:

> You are referring to something JavaScript does not know.

7.1 Example: Variable Was Never Declared

Incorrect:

```
console.log(username);
```

Possible error:

```
ReferenceError: username is not defined
```

Correct:

```
const username = "Ada";
console.log(username);
```

7.2 Example: Misspelled Variable Name

Incorrect:

```
const userName = "Ada";

console.log(username);
```

Problem:

```
userName and username are different names.
```

Correct:

```
const userName = "Ada";

console.log(userName);
```

7.3 Example: Using `let` or `const` Too Early

Incorrect:

```
console.log(score);

let score = 10;
```

This can cause an error because `let` and `const` variables cannot be used before they are initialized.

Correct:

```
let score = 10;

console.log(score);
```

7.4 Common Causes of `ReferenceError`

A `ReferenceError` often happens because:

- a variable was never declared
- a variable name was misspelled
- code is running before a variable exists
- a variable is outside the current scope

Beginner rule:

If you see `ReferenceError`, check the spelling, declaration, and scope of the variable.

---

8. `TypeError`

A `TypeError` happens when a value exists, but you use it in a way that does not fit its type.

Simple meaning:

> The value is the wrong kind of value for the operation.

8.1 Example: Calling `.map()` on an Object

Incorrect:

```
const user = {
  name: "Ada",
  score: 90
};

user.map(item => item);
```

Possible error:

```
TypeError: user.map is not a function
```

Why?

`user` is a plain object.

`.map()` works on arrays, not plain objects.

Correct:

```
const users = [
  { name: "Ada", score: 90 },
  { name: "Grace", score: 85 }
];

const names = users.map(user => user.name);

console.log(names);
```

Output:

```
["Ada", "Grace"]
```

8.2 Example: Using a Method on `null`

Incorrect:

```
const button = document.getElementById("missingButton");

button.addEventListener("click", () => {
  console.log("Clicked");
});
```

If no element has the ID `missingButton`, then `button` is `null`.

Possible error:

```
TypeError: Cannot read properties of null
```

Safer version:

```
const button = document.getElementById("missingButton");

if (button !== null) {
  button.addEventListener("click", () => {
    console.log("Clicked");
  });
}
```

8.3 Example: Calling Something That Is Not a Function

Incorrect:

```
const message = "Hello";

message();
```

Possible error:

```
TypeError: message is not a function
```

Why?

`message` is a string, not a function.

8.4 Common Causes of `TypeError`

A `TypeError` often happens because:

- you call an array method on a non-array
- you call a function that is not actually a function
- you read a property from `null`
- you read a property from `undefined`
- you use a value before checking its type

Beginner rule:

> If you see `TypeError`, check what type the value really is.

Useful check:

```
console.log(typeof user);
console.log(Array.isArray(user));
```

---

9. `RangeError`

A `RangeError` happens when a value is outside the allowed range.

Simple meaning:

> The value has the right general type, but the value is not allowed.

9.1 Example: Invalid Array Length

Incorrect:

```
const items = [];

items.length = -1;
```

Possible error:

```
RangeError: Invalid array length
```

Why?

An array length cannot be negative.

Correct:

```
const items = [];

items.length = 0;
```

9.2 Example: Custom Range Rule

You may want an age to be between `0` and `120`.

```
function validateAge(age) {
  if (age < 0 || age > 120) {
    throw new RangeError("Age must be between 0 and 120");
  }

  return true;
}
```

9.3 Common Causes of `RangeError`

A `RangeError` often happens because:

- a number is too small
- a number is too large
- an array length is invalid
- a function receives a value outside its allowed range

Beginner rule:

> If you see `RangeError`, check the allowed minimum and maximum values.

---

10. Comparison of Common Error Types

Error Type	Simple Meaning	Common Cause
`SyntaxError`	JavaScript cannot understand the code.	Missing bracket, quote, parenthesis, or invalid punctuation.
`ReferenceError`	JavaScript cannot find a name.	Variable was not declared, misspelled, or out of scope.
`TypeError`	The value is the wrong type for the operation.	Calling `.map()` on an object, reading from `null`, calling a non-function.
`RangeError`	The value is outside the allowed range.	Negative array length, invalid number range.

---

11. Throwing Your Own Errors

Sometimes JavaScript will not automatically complain, but your app should still reject bad input.

This is where `throw` is useful.

11.1 What `throw` Does

`throw` creates an error intentionally.

Simple meaning:

> Stop normal execution because something is wrong.

Example:

```
throw new Error("Something went wrong");
```

11.2 Example: Division by Zero

JavaScript allows this:

```
function divide(a, b) {
  return a / b;
}

console.log(divide(10, 0));
```

Output:

```
Infinity
```

But in many apps, dividing by zero should be treated as invalid.

Better version:

```
function divide(a, b) {
  if (b === 0) {
    throw new Error("Cannot divide by zero");
  }

  return a / b;
}

console.log(divide(10, 2));
console.log(divide(10, 0));
```

Output:

```
5
Error: Cannot divide by zero
```

11.3 When to Throw an Error

Use `throw` when:

- the input is invalid
- continuing would create a wrong result
- a function cannot do its job safely
- a required value is missing
- a value has the wrong type
- a value is outside the allowed range

Beginner rule:

> Throw an error when continuing would hide a real problem.

---

12. `Error`, `TypeError`, and `RangeError`

You can throw different kinds of errors.

12.1 General `Error`

Use `Error` for a general problem.

```
throw new Error("Something went wrong");
```

Example:

```
function requireUsername(username) {
  if (!username) {
    throw new Error("Username is required");
  }

  return username;
}
```

12.2 `TypeError`

Use `TypeError` when a value has the wrong type.

```
throw new TypeError("Price must be a number");
```

Example:

```
function validatePrice(price) {
  if (typeof price !== "number") {
    throw new TypeError("Price must be a number");
  }

  return true;
}
```

12.3 `RangeError`

Use `RangeError` when a value is outside the allowed range.

```
throw new RangeError("Age must be between 0 and 120");
```

Example:

```
function validateAge(age) {
  if (age < 0 || age > 120) {
    throw new RangeError("Age must be between 0 and 120");
  }

  return true;
}
```

12.4 Full Example

```
function validateAge(age) {
  if (typeof age !== "number") {
    throw new TypeError("Age must be a number");
  }

  if (age < 0 || age > 120) {
    throw new RangeError("Age must be between 0 and 120");
  }

  return true;
}

console.log(validateAge(16));
console.log(validateAge("16"));
console.log(validateAge(200));
```

What happens:

```
validateAge(16) returns true.
validateAge("16") throws TypeError.
validateAge(200) throws RangeError.
```

---

13. `try...catch...finally`

`try...catch...finally` lets you handle errors without letting the whole app crash immediately.

Basic shape:

```
try {
  // risky code
} catch (error) {
```

```
  // handle the error
} finally {
  // always run this code
}
```

---

14. `try`

`try` contains code that might fail.

Example:

```
try {
  const data = JSON.parse("not valid json");
  console.log(data);
} catch (error) {
  console.log("Parsing failed");
}
```

Simple meaning:

> Try to run this risky code.

---

15. `catch`

`catch` runs if an error happens inside `try`.

Example:

```
try {
  JSON.parse("not valid json");
} catch (error) {
  console.log("There was an error");
  console.log(error.message);
}
```

Simple meaning:

> If the risky code fails, handle the error here.

The variable `error` receives the error object.

You can name it differently, but `error` is clear for beginners.

---

16. `finally`

`finally` runs after `try` and `catch`, no matter what.

Example:

```
try {
  console.log("Trying...");
} catch (error) {
  console.log("Error happened");
} finally {
  console.log("Finished checking");
}
```

Output:

```
Trying...
Finished checking
```

If an error happens, `finally` still runs.

Example:

```
try {
  throw new Error("Problem");
} catch (error) {
  console.log(error.message);
} finally {
  console.log("Finished checking");
}
```

Output:

```
Problem
Finished checking
```

Simple meaning:

> `finally` is for cleanup or code that must always happen.

17. Full `try...catch...finally` Example

```
function divide(numerator, denominator) {
  if (denominator === 0) {
    throw new Error("Cannot divide by zero");
  }

  return numerator / denominator;
}

try {
  console.log(divide(10, 0));
} catch (error) {
  console.log(error.message);
} finally {
  console.log("Done checking division.");
}
```

Output:

```
Cannot divide by zero
Done checking division.
```

Step by step:

1. JavaScript enters the `try` block.
2. `divide(10, 0)` runs.
3. The denominator is `0`.
4. The function throws an error.
5. JavaScript jumps to `catch`.
6. `catch` receives the error object.
7. `error.message` is printed.
8. `finally` runs.

---

18. When Should You Use `try...catch`?

Use `try...catch` when code can realistically fail.

Good examples:

18.1 Parsing JSON

```
JSON.parse(savedData);
```

This can fail if `savedData` is not valid JSON.

18.2 Loading Browser Storage

```
const tasks = JSON.parse(localStorage.getItem("tasks"));
```

This can fail if the saved data is broken or manually changed.

18.3 Network Requests

```
fetch(url);
```

Network requests can fail because of internet problems, server problems, or invalid URLs.

18.4 Risky Calculations

```
divide(10, 0);
```

Some inputs may be invalid.

18.5 User Input

```
validateNumber(input);
```

User input may have the wrong type or format.

Beginner rule:

> Use `try...catch` when failure is possible and you have a plan for what to do if it fails.

---

19. Safe Loading Example from Browser Storage

When you load saved data from `localStorage`, do not blindly trust it.

Risky version:

```
const tasks = JSON.parse(localStorage.getItem("tasks"));
```

Problem:

```
If the saved data is broken, JSON.parse() can throw an error.
```

Safer version:

```
function safeLoadTasks() {
  try {
    const saved = JSON.parse(localStorage.getItem("tasks")) || [];

    if (Array.isArray(saved)) {
      return saved;
    }

    return [];
  } catch (error) {
    return [];
  }
}
```

What this does:

1. Tries to parse saved task data.
2. If parsing works, checks whether the result is an array.
3. If the result is an array, returns it.
4. If the result is not an array, returns an empty array.
5. If parsing fails, returns an empty array.

Simple meaning:

> The app tries to load saved tasks, but it falls back safely if the saved data is invalid.

---

20. The `error` Object

Inside `catch`, JavaScript gives you an error object.

Example:

```
try {
  JSON.parse("not valid json");
} catch (error) {
  console.log(error.name);
  console.log(error.message);
}
```

Possible output:

```
SyntaxError
Unexpected token ...
```

20.1 `error.name`

`error.name` gives the type of error.

Examples:

```
SyntaxError
ReferenceError
TypeError
RangeError
```

20.2 `error.message`

`error.message` gives the readable explanation.

Example:

```
try {
  throw new Error("Cannot divide by zero");
} catch (error) {
  console.log(error.message);
}
```

Output:

```
Cannot divide by zero
```

20.3 `error.stack`

`error.stack` gives more detailed information about where the error happened.

Example:

```
try {
  throw new Error("Problem");
} catch (error) {
  console.log(error.stack);
}
```

The exact output depends on the browser.

Simple meaning:

- `error.stack` helps you trace where the error came from.

20.4 Useful Error Logging Pattern

```
try {
  riskyCode();
} catch (error) {
  console.error(error.name);
  console.error(error.message);
}
```

Use `console.error()` when you want the console to clearly show an error.

---

21. `console.log()` for Debugging

`console.log()` prints values in the console.

Example:

```
const price = "abc";
const quantity = 3;

console.log("price:", price);
console.log("quantity:", quantity);
console.log("typeof price:", typeof price);
console.log("typeof quantity:", typeof quantity);
```

```
const total = price * quantity;

console.log("total:", total);
```

Output:

```
price: abc
typeof price: string
quantity: 3
typeof quantity: number
total: NaN
```

This helps you see where the wrong value is.

Beginner rule:

> Log the value and the type, not only the value.

Useful pattern:

```
console.log("value:", value);
console.log("type:", typeof value);
```

For arrays:

```
console.log(Array.isArray(value));
```

---

22. `console.error()`

`console.error()` prints an error-style message in the console.

Example:

```
try {
  throw new Error("Something failed");
} catch (error) {
  console.error(error.message);
}
```

Output:

```
Something failed
```

Simple meaning:

> Use `console.error()` for error messages that should stand out.

---

23. `console.table()`

`console.table()` prints arrays or objects in table form.

This is useful when debugging structured data.

23.1 Example with Array of Objects

```
const tasks = [
  { text: "Study Sprint 18", done: false },
  { text: "Practice debugging", done: true },
  { text: "Review errors", done: false }
];

console.table(tasks);
```

This is easier to inspect than:

```
console.log(tasks);
```

23.2 Example with Users

```
const users = [
  { name: "Ada", score: 90 },
  { name: "Grace", score: 85 },
  { name: "Linus", score: 95 }
];

console.table(users);
```

Good uses for `console.table()`:

- arrays of objects

- task lists
- user lists
- product lists
- rows of data
- checking before and after updates

Beginner rule:

> Use `console.table()` when the data has rows and columns.

---

24. `debugger`

The `debugger` statement pauses your code when browser DevTools is open.

Example:

```
function calculateTotal(price, quantity) {
  debugger;

  const total = price * quantity;

  return total;
}

console.log(calculateTotal(10, 3));
```

When the browser reaches `debugger`, it pauses the code.

Then you can inspect values like:

```
price
quantity
total
```

24.1 How to Use `debugger`

1. Open browser DevTools.
2. Go to the Sources tab or Console tab.
3. Run your code.
4. JavaScript pauses at the `debugger` line.
5. Inspect variables.
6. Step through the code line by line.

Simple meaning:

`debugger` lets you stop the program and look inside it while it is running.

24.2 When to Use `debugger`

Use `debugger` when:

- `console.log()` is not enough
- a value changes across several lines
- a loop is producing the wrong result
- a function receives unexpected input
- DOM code is running in the wrong order
- storage data is not loading correctly

Beginner rule:

> Use `debugger` when you need to pause and inspect instead of only printing values.

---

25. Browser DevTools

Browser DevTools are tools built into the browser for developers.

You can usually open DevTools by:

```
Right click page -> Inspect
```

Or with keyboard shortcuts such as:

```
Ctrl + Shift + I
Ctrl + Shift + J
F12
```

The exact shortcut depends on your computer and browser.

Useful DevTools areas:

Area	Use
Console	See logs and errors.
Sources	Pause code, add breakpoints, step through code.
Elements	Inspect HTML and CSS.
Application	Inspect storage like `localStorage` and `sessionStorage`.

Area	Use
Network	Inspect network requests.
Performance	Investigate slow code.

Sprint 18 mainly focuses on:

- Console
- Sources
- Breakpoints
- Watch expressions
- Basic profiling awareness

---

26. Breakpoints

A breakpoint pauses code at a specific line in DevTools.

Simple meaning:

> A breakpoint is like adding `debugger`, but you do it in DevTools instead of writing it in your code.

26.1 Example Code

```
function updateTask(index, newText) {
  tasks[index].text = newText;
  saveTasks();
}
```

You may want to pause on this line:

```
tasks[index].text = newText;
```

Then inspect:

```
index
newText
tasks[index]
tasks
```

26.2 How to Set a Breakpoint

1. Open DevTools.
2. Go to the Sources tab.
3. Open your JavaScript file.
4. Click the line number where you want to pause.
5. Trigger the code in the page.
6. The browser pauses on that line.
7. Inspect values.
8. Step through the code.

26.3 When to Use Breakpoints

Use breakpoints when:

- you do not want to edit the source code
- you want to pause on a specific line
- you want to inspect values during event handlers
- you want to inspect loop behavior
- you want to inspect DOM updates
- you want to inspect storage updates

Beginner rule:

> Breakpoints are better than many random `console.log()` statements when the logic is complex.

---

27. Stepping Through Code

When code is paused, DevTools usually gives you buttons for stepping.

The exact names can vary by browser, but the idea is similar.

Action	Simple Meaning
Step over	Run the current line and move to the next line.
Step into	Go inside the function being called.
Step out	Finish the current function and return to where it was called.
Resume	Continue running normally until the next breakpoint.

Example:

```
function getTotal(price, quantity) {
  return price * quantity;
}

function showTotal() {
  const total = getTotal(10, 3);
  console.log(total);
}

showTotal();
```

If paused at this line:

```
const total = getTotal(10, 3);
```

- Step over runs `getTotal()` without going inside it.
- Step into enters the `getTotal()` function.

Beginner rule:

> Step into a function when you suspect the function itself is wrong.

---

28. Watch Expressions

A watch expression is a value you ask DevTools to keep showing while the code is paused.

Simple meaning:

> A watch expression lets you monitor important values while debugging.

28.1 Example

```
function updateTask(index, newText) {
  tasks[index].text = newText;
  saveTasks();
}
```

Useful watch expressions:

```
index
newText
```

```
tasks[index]
tasks.length
```

28.2 More Watch Expression Examples

```
typeof input
Number(input)
Array.isArray(tasks)
tasks.length
user.name
button === null
```

28.3 When to Use Watch Expressions

Use watch expressions when:

- the same value matters across several lines
- you want to see how a value changes
- you are inside a loop
- you are inside an event handler
- you are debugging a function with several steps

Beginner rule:

> Watch the values that decide what the code does next.

---

29. Profiling and Performance Awareness

Profiling means measuring how code behaves while it runs.

Beginner meaning:

> Profiling helps you find slow or expensive code.

You do not need advanced performance work yet, but you should know the idea.

29.1 Simple Timing with `Date.now()`

```
const start = Date.now();

for (let i = 0; i < 1000000; i++) {
  // repeated work
}
```

```
const end = Date.now();

console.log(`Time passed: ${end - start} ms`);
```

This tells you roughly how long the code took.

29.2 When Profiling Matters

Profiling matters when:

- a page feels slow
- a loop processes a lot of data
- DOM updates happen many times
- an event handler runs too often
- animation feels rough
- storage or rendering feels delayed

Beginner rule:

> First make the code correct. Then measure if it feels slow.

---

30. Beginner Testing with `console.assert()`

Testing means checking that your code gives the expected result.

`console.assert()` is a beginner-friendly way to test simple functions.

30.1 Basic Syntax

```
console.assert(condition, "Message if condition is false");
```

If the condition is true, usually nothing happens.

If the condition is false, the console shows the message.

30.2 Example

```
function add(a, b) {
  return a + b;
}

console.assert(add(2, 3) === 5, "add(2, 3) should be 5");
```

```
console.assert(add(-1, 1) === 0, "add(-1, 1) should be 0");
console.assert(add(0, 0) === 0, "add(0, 0) should be 0");
```

These pass because the function returns the expected values.

30.3 Example Failure

```
console.assert(add(2, 3) === 6, "Expected 6");
```

This fails because:

```
add(2, 3)
```

returns:

```
5
```

not:

```
6
```

30.4 Why `console.assert()` Is Useful

It helps you check:

- normal cases
- edge cases
- zero values
- empty strings
- invalid values
- expected outputs

Beginner rule:

> Use `console.assert()` to prove that small functions behave correctly.

---

31. Testing Expected Outputs and Edge Cases

An expected output is the result you think the function should return.

Example:

```
function double(number) {
  return number * 2;
}
```

Expected outputs:

```
double(2) === 4
double(0) === 0
double(-3) === -6
```

Tests:

```
console.assert(double(2) === 4, "2 doubled should be 4");
console.assert(double(0) === 0, "0 doubled should be 0");
console.assert(double(-3) === -6, "-3 doubled should be -6");
```

31.1 What Is an Edge Case?

An edge case is an unusual or boundary input.

Examples:

- `0`
- negative numbers
- empty strings
- empty arrays
- `null`
- `undefined`
- very large numbers
- invalid input type

Example:

```
function getFirstItem(items) {
  return items[0];
}
```

Normal case:

```
getFirstItem(["a", "b"]);
```

Edge case:

```
getFirstItem([]);
```

The result is:

```
undefined
```

Beginner rule:

> Test both normal inputs and edge cases.

---

32. Mini-Build: `validateNumber(input)`

The Sprint 18 mini-build is a function that validates numbers.

Goal:

> Write a function that throws `TypeError` for non-numbers.

32.1 First Version

```
function validateNumber(input) {
  if (typeof input !== "number") {
    throw new TypeError("Input must be a number");
  }

  return input;
}

try {
  console.log(validateNumber(10));
  console.log(validateNumber("10"));
} catch (error) {
  console.error(error.name);
  console.error(error.message);
}
```

Output:

```
10
TypeError
Input must be a number
```

32.2 Why This Works

This line checks the type:

```
typeof input !== "number"
```

If the input is not a number, the function throws a `TypeError`.

32.3 Problem: `NaN`

This is tricky:

```
typeof NaN
```

Output:

```
number
```

That means this check is not enough:

```
typeof input !== "number"
```

Because `NaN` still has type `number`.

32.4 Better Version: Reject `NaN`

```
function validateNumber(input) {
  if (typeof input !== "number") {
    throw new TypeError("Input must be a number");
  }

  if (Number.isNaN(input)) {
    throw new TypeError("Input cannot be NaN");
  }

  return input;
```

```
}

console.log(validateNumber(25));
```

Output:

```
25
```

32.5 Test Cases

```
console.assert(validateNumber(10) === 10, "10 should be valid");
console.assert(validateNumber(0) === 0, "0 should be valid");
console.assert(validateNumber(-5) === -5, "-5 should be valid");
```

Do not test throwing cases with plain `console.assert()` unless you wrap them carefully in `try...catch`.

---

33. Full Practice: Safe Calculator

This example combines:

- validation
- `TypeError`
- `Error`
- `throw`
- `try`
- `catch`
- `finally`

```
function validateNumber(value, name) {
  if (typeof value !== "number") {
    throw new TypeError(`${name} must be a number`);
  }

  if (Number.isNaN(value)) {
    throw new TypeError(`${name} cannot be NaN`);
  }
}

function divide(a, b) {
  validateNumber(a, "a");
  validateNumber(b, "b");
```

```
  if (b === 0) {
    throw new Error("Cannot divide by zero");
  }

  return a / b;
}

try {
  const result = divide(10, 2);
  console.log(result);
} catch (error) {
  console.error(error.name);
  console.error(error.message);
} finally {
  console.log("Division check finished");
}
```

Expected output:

```
5
Division check finished
```

33.1 Invalid Cases

Test invalid cases one by one.

```
divide("10", 2); // TypeError
divide(10, "2"); // TypeError
divide(10, 0);   // Error
divide(NaN, 2);  // TypeError
```

Do not run all of these together unless each one is wrapped in its own `try...catch`.

Why?

Because the first thrown error stops the normal flow.

---

34. Debugging Wrong Data Types

A common beginner bug is using the wrong data type.

Example:

```
const developerObj = {
  name: "Sangeeth",
  language: "JavaScript"
};

const result = developerObj.map(item => item);

console.log(result);
```

Problem:

```
developerObj is an object.
.map() belongs to arrays.
```

Possible error:

```
TypeError: developerObj.map is not a function
```

34.1 How to Debug It

Check the value:

```
console.log(developerObj);
```

Check the type:

```
console.log(typeof developerObj);
```

Check whether it is an array:

```
console.log(Array.isArray(developerObj));
```

Output:

```
object
false
```

34.2 Correct Version

Use an array if you want `.map()`:

```
const developers = [
  { name: "Sangeeth", language: "JavaScript" }
];

const names = developers.map(developer => developer.name);

console.log(names);
```

Output:

```
["Sangeeth"]
```

Beginner rule:

> Before using an array method, confirm that the value is actually an array.

---

35. Debugging `null` DOM Elements

Another common beginner bug happens when JavaScript cannot find an HTML element.

HTML:

```
<button id="saveBtn">Save</button>
```

JavaScript:

```
const button = document.getElementById("saveButton");

button.addEventListener("click", () => {
  console.log("Saved");
});
```

Problem:

```
The HTML id is saveBtn.
The JavaScript searches for saveButton.
```

So this returns `null`:

```
document.getElementById("saveButton")
```

Possible error:

```
TypeError: Cannot read properties of null
```

35.1 Debugging Steps

```
const button = document.getElementById("saveButton");

console.log(button);
```

If output is:

```
null
```

Then JavaScript did not find the element.

35.2 Correct Version

```
const button = document.getElementById("saveBtn");

button.addEventListener("click", () => {
  console.log("Saved");
});
```

35.3 Safer Version

```
const button = document.getElementById("saveBtn");

if (button !== null) {
  button.addEventListener("click", () => {
    console.log("Saved");
```

```
  });
}
```

Beginner rule:

> If DOM code says it cannot read from `null`, check your selector and script timing.

---

36. Debugging Script Timing Problems

Sometimes JavaScript runs before the HTML element exists.

Problem example:

```
<script>
  const button = document.getElementById("runBtn");

  button.addEventListener("click", () => {
    console.log("Clicked");
  });
</script>

<button id="runBtn">Run</button>
```

Problem:

```
The script runs before the button exists in the DOM.
```

So `button` is `null`.

36.1 Fix 1: Move Script to End of Body

```
<button id="runBtn">Run</button>

<script>
  const button = document.getElementById("runBtn");

  button.addEventListener("click", () => {
    console.log("Clicked");
  });
</script>
```

36.2 Fix 2: Use `DOMContentLoaded`

```
document.addEventListener("DOMContentLoaded", () => {
  const button = document.getElementById("runBtn");

  button.addEventListener("click", () => {
    console.log("Clicked");
  });
});
```

Simple meaning:

> Wait until the HTML has been loaded before selecting elements.

---

37. Debugging Stored Data

Stored browser data can be wrong.

Example:

```
const tasks = JSON.parse(localStorage.getItem("tasks"));

tasks.forEach(task => {
  console.log(task.text);
});
```

Possible problem:

```
tasks might be null.
tasks might not be an array.
JSON.parse() might fail.
```

Safer version:

```
function loadTasks() {
  try {
    const saved = JSON.parse(localStorage.getItem("tasks"));

    if (Array.isArray(saved)) {
      return saved;
    }
```

```
    return [];
  } catch (error) {
    console.error("Could not load tasks:", error.message);
    return [];
  }
}

const tasks = loadTasks();

console.table(tasks);
```

Beginner rule:

> Treat stored data as untrusted. Check it before using it.

---

38. Debugging Checklist

When something breaks, follow this checklist.

38.1 Error Message Checklist

1. What is the error name?
2. What is the error message?
3. What file is mentioned?
4. What line number is mentioned?
5. Does the error point to the real cause or only where the code failed?

38.2 Value Checklist

1. What is the value?
2. What is its type?
3. Is it `null`?
4. Is it `undefined`?
5. Is it an array?
6. Is it an object?
7. Is it a number?
8. Is it `NaN`?

Useful checks:

```
console.log(value);
console.log(typeof value);
console.log(value === null);
console.log(value === undefined);
```

```
console.log(Array.isArray(value));
console.log(Number.isNaN(value));
```

38.3 Function Checklist

1. Did the function run?
2. What arguments did it receive?
3. What did it return?
4. Did it throw an error?
5. Did it change external data?

Useful pattern:

```
function example(input) {
  console.log("input:", input);
  console.log("type:", typeof input);

  const result = input;

  console.log("result:", result);

  return result;
}
```

38.4 DOM Checklist

1. Does the element exist in the HTML?
2. Is the selector spelled correctly?
3. Is the script running before the HTML exists?
4. Is the selected value `null`?
5. Are you using the right property or method?

Useful check:

```
const element = document.getElementById("title");
console.log(element);
```

38.5 Storage Checklist

1. Does the key exist in `localStorage`?
2. Is the stored value valid JSON?
3. Is the parsed value the expected type?
4. Should the fallback be `[]`, `{}`, `null`, or another value?
5. Are you saving after every change?

Useful check:

```
console.log(localStorage.getItem("tasks"));
```

---

39. Practice Task 1: Create Four Errors on Purpose

Run each example separately.

39.1 Create a `SyntaxError`

```
console.log("Hello";
```

Question:

> What punctuation is missing?

Answer:

```
A closing parenthesis is missing.
```

Correct:

```
console.log("Hello");
```

39.2 Create a `ReferenceError`

```
console.log(missingVariable);
```

Question:

> Why does this fail?

Answer:

```
missingVariable was never declared.
```

Correct:

```
const missingVariable = "Now I exist";
console.log(missingVariable);
```

39.3 Create a `TypeError`

```
const user = { name: "Ada" };

user.map(item => item);
```

Question:

> Why does this fail?

Answer:

```
user is an object, not an array. Plain objects do not have .map().
```

Correct:

```
const users = [{ name: "Ada" }];

const names = users.map(user => user.name);

console.log(names);
```

39.4 Create a `RangeError`

```
const items = [];

items.length = -1;
```

Question:

> Why does this fail?

Answer:

```
Array length cannot be negative.
```

Correct:

```
const items = [];

items.length = 0;
```

---

40. Practice Task 2: Debug This Object Problem

Broken code:

```
const developerObj = {
  name: "Sangeeth",
  language: "JavaScript"
};

const result = developerObj.map(item => item);

console.log(result);
```

40.1 What Is Wrong?

`developerObj` is a plain object.

`.map()` belongs to arrays.

So JavaScript throws a `TypeError`.

40.2 How to Prove It

```
console.log(typeof developerObj);
console.log(Array.isArray(developerObj));
```

Expected output:

```
object
false
```

40.3 Correct Version

```
const developers = [
  { name: "Sangeeth", language: "JavaScript" }
];
```

```
const names = developers.map(developer => developer.name);

console.log(names);
```

Expected output:

```
["Sangeeth"]
```

---

41. Practice Task 3: Use `console.table()`

```
const tasks = [
  { id: 1, text: "Learn errors", done: false },
  { id: 2, text: "Use debugger", done: false },
  { id: 3, text: "Practice try/catch", done: true }
];

console.table(tasks);
```

Questions:

1. How many tasks are there?
2. Which task is done?
3. Which tasks are not done?
4. What are the object property names?

Expected answers:

```
There are 3 tasks.
Task 3 is done.
Tasks 1 and 2 are not done.
The properties are id, text, and done.
```

---

42. Practice Task 4: Add Tests with `console.assert()`

```
function addTax(price, rate) {
  return price + price * rate;
}
```

```
console.assert(addTax(100, 0.1) === 110, "100 with 10% tax should be 110");
console.assert(addTax(0, 0.1) === 0, "0 with tax should be 0");
console.assert(addTax(50, 0) === 50, "50 with 0% tax should be 50");
```

What this checks:

1. Normal case: `100` with `10%` tax.
2. Edge case: `0` price.
3. Edge case: `0%` tax.

Beginner rule:

> A good test checks both normal cases and edge cases.

---

43. Practice Task 5: Debug a Safe Division Function

Broken version:

```
function divide(a, b) {
  return a / b;
}

console.log(divide(10, 0));
console.log(divide("10", 2));
```

Problems:

```
Dividing by zero returns Infinity.
String input may be accepted accidentally.
```

Improved version:

```
function validateNumber(value, name) {
  if (typeof value !== "number") {
    throw new TypeError(`${name} must be a number`);
  }

  if (Number.isNaN(value)) {
    throw new TypeError(`${name} cannot be NaN`);
  }
}
```

```
function divide(a, b) {
  validateNumber(a, "a");
  validateNumber(b, "b");

  if (b === 0) {
    throw new Error("Cannot divide by zero");
  }

  return a / b;
}
```

Test:

```
try {
  console.log(divide(10, 2));
} catch (error) {
  console.error(error.name);
  console.error(error.message);
}
```

Expected output:

```
5
```

---

44. Common Sprint 18 Beginner Mistakes

Mistake 1: Ignoring the Error Message

Weak approach:

```
The code is broken. I do not know why.
```

Better approach:

```
The error is a TypeError on line 12. It says .map is not a function. I should
check whether the value is an array.
```

Mistake 2: Only Logging the Value, Not the Type

Weak:

```
console.log(input);
```

Better:

```
console.log(input);
console.log(typeof input);
```

Mistake 3: Calling Array Methods on Objects

Incorrect:

```
const user = { name: "Ada" };
user.map(item => item);
```

Correct:

```
const users = [{ name: "Ada" }];
users.map(user => user.name);
```

Mistake 4: Assuming DOM Selection Always Works

Incorrect:

```
const button = document.getElementById("btn");
button.addEventListener("click", handleClick);
```

Safer:

```
const button = document.getElementById("btn");

if (button !== null) {
  button.addEventListener("click", handleClick);
}
```

Mistake 5: Trusting Stored Data Without Checking

Incorrect:

```
const tasks = JSON.parse(localStorage.getItem("tasks"));
tasks.forEach(task => console.log(task.text));
```

Safer:

```
const tasks = safeLoadTasks();
console.table(tasks);
```

Mistake 6: Catching Errors but Hiding Everything

Weak:

```
try {
  riskyCode();
} catch (error) {
}
```

Problem:

```
The error disappears, and you do not know what went wrong.
```

Better:

```
try {
  riskyCode();
} catch (error) {
  console.error(error.name);
  console.error(error.message);
}
```

Mistake 7: Using `try...catch` Instead of Fixing Bad Logic

`try...catch` is not a replacement for clear logic.

Weak:

```
try {
  button.addEventListener("click", handleClick);
} catch (error) {
```

```
  console.error(error.message);
}
```

Better:

```
if (button !== null) {
  button.addEventListener("click", handleClick);
}
```

Beginner rule:

> Use `try...catch` for realistic failures, not as a way to avoid basic checks.

---

45. Sprint 18 Mental Models

45.1 Error Message Mental Model

```
Error message = clue
Error name = category
Line number = starting point
```

45.2 Debugging Mental Model

```
Expected result
↓
Actual result
↓
Find wrong value
↓
Find where it became wrong
↓
Fix one cause
↓
Test again
```

45.3 `try...catch` Mental Model

```
try = run risky code
catch = handle failure
finally = always run cleanup
```

45.4 `throw` Mental Model

```
Bad input found
↓
Throw error
↓
Stop normal flow
↓
Let catch handle it
```

45.5 DevTools Mental Model

```
console.log = print values
debugger = pause code
breakpoint = pause code without editing source
watch expression = monitor values
console.table = inspect rows
console.assert = check expected output
```

---

46. Sprint 18 Review Questions

You should be able to answer these.

1. What does debugging mean?
2. What are the three main parts of an error message?
3. What is a `SyntaxError`?
4. What is a `ReferenceError`?
5. What is a `TypeError`?
6. What is a `RangeError`?
7. Why does calling `.map()` on a plain object cause a `TypeError`?
8. What does `throw` do?
9. When should you throw your own error?
10. What is the difference between `Error`, `TypeError`, and `RangeError`?
11. What does `try` do?
12. What does `catch` do?
13. What does `finally` do?
14. What is the `error` object?
15. What is `error.name`?
16. What is `error.message`?
17. What is `error.stack`?
18. When should you use `try...catch`?
19. Why can `JSON.parse()` fail?
20. Why should stored browser data be checked before use?

21. What does `debugger` do?
22. What is a breakpoint?
23. What is a watch expression?
24. When should you use `console.table()`?
25. What does `console.assert()` do?
26. Why should you test edge cases?
27. Why is `typeof NaN` confusing?
28. How do you check whether a value is an array?
29. How do you check whether a number is `NaN`?
30. What should you do first when code breaks?

---

47. Minimum Passing Standard for Sprint 18

You are ready for Sprint 19 when you can explain these without looking:

1. `SyntaxError` means JavaScript cannot understand the code structure.
2. `ReferenceError` means JavaScript cannot find a variable or name.
3. `TypeError` means a value is being used in a way that does not fit its type.
4. `RangeError` means a value is outside the allowed range.
5. `.map()` works on arrays, not plain objects.
6. `throw` creates an error intentionally.
7. `try` runs risky code.
8. `catch` handles an error from `try`.
9. `finally` runs no matter what.
10. The `error` object contains information about what went wrong.
11. `debugger` pauses code when DevTools is open.
12. A breakpoint pauses code at a selected line in DevTools.
13. A watch expression monitors a value while code is paused.
14. `console.table()` displays row-like data clearly.
15. `console.assert()` checks whether an expected condition is true.
16. `JSON.parse()` can fail if the string is not valid JSON.
17. Stored data should be checked before use.
18. Debugging should be systematic, not random.

---

48. Final Sprint 18 Memory Card

```
Error message = clue, not enemy.

Read:
1. Error name
2. Error message
3. File and line number
```

```
Then inspect:
1. Value
2. Type
3. null / undefined
4. array or not array
5. expected output vs actual output

Use:
console.log()      -> print simple values
console.error()    -> print error messages
console.table()    -> inspect rows of data
debugger           -> pause code
breakpoints        -> pause code in DevTools
watch expressions  -> monitor values
console.assert()   -> test expected results
try...catch        -> handle realistic failures
throw              -> create useful errors
```

Core idea:

> Sprint 18 is not mainly about writing more code. It is about learning how to understand code when it fails.